P05: Le Fin

Design Doc by PortendedGreatness, pd. 5

↳ *Roster: Yuhang Pan (PM), Andrew Tsai, Owen Zeng, Zixi Qiao*

PROJECT NAME: Copyrighted Artists
DESCRIPTION: The initial load page will lead you to register an account. Afterwards, you will be redirected to a homepage, from which you can join game lobbies. Upon joining a game lobby, you will be led to a game window where you can draw based on a prompt or based off of what other people have drawn. The site will award you elo points based on winning or losing.

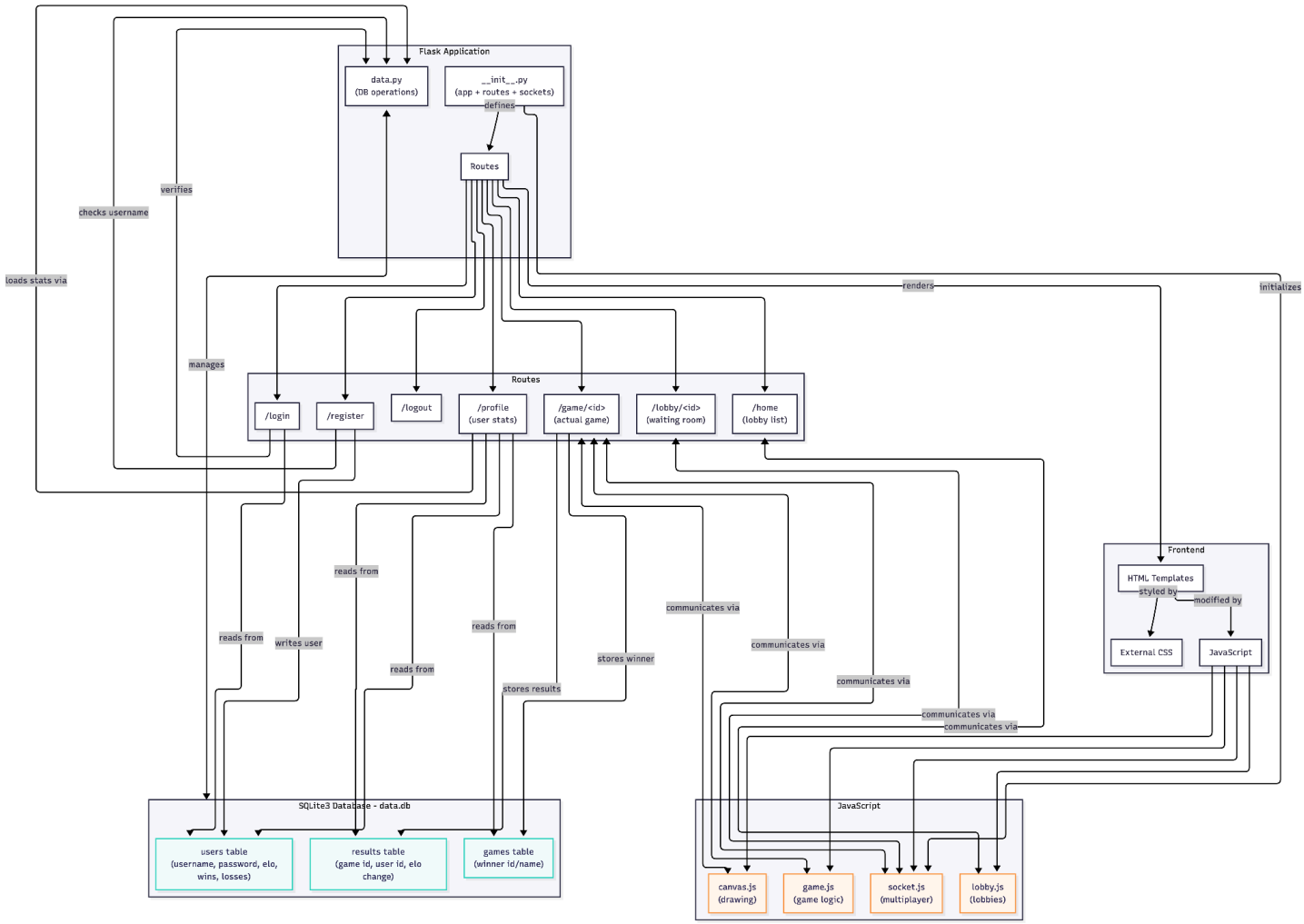
TARGET SHIP DATE: 06-07-2026

Program Components:

1) Flask App (Python)
    a) **__init__.py** (main app + routes)
        i) Creates Flask app, config, and session handling
        ii) Renders templates and connects frontend to backend logic
        iii) Initialize sockets for multiplayer
        iv) Routes:
            (1) **/register** adds a new user to the **users** table (**data.py**). Checks if the username is unique, hashes the password, stores it in session, then redirects to **/home**.
            (2) **/login** checks users (**data.py**), verifies password hash, stores session, and redirects to **/home**.
            (3) **/logout** clears session, redirects to **/login**.
            (4) **/home** lobby list where users can join or create a lobby
            (5) **/lobby/<id>** waiting room for players to fill up to start the game
            (6) **/game/<id>** the actual game
            (7) **/profile** uses **data.py** to load a user's stats
    b) **data.py**
        i) Connects to SQLite3 database and creates/maintains tables
2) Database (SQLite3) (stored in data.db)
    a) **users** table stores all usernames and password hashes for authentication, and other stats like elo, #wins, #losses, etc.
    b) **games** table stores the winner's id/name to be used for profile page
    c) **results** table stores the a pair of game&user id to keep track of a user's game history like elo change in the game played
3) Frontend Framework
    a) **?**
4) JavaScript
    a) **lobby.js** handles the lobbies
    b) **game.js** handles the game stuff
    c) **canvas.js** handles the drawing on js canvas
    d) **socket.js** handles the sockets for multiplayer
5) RESTful APIs
    a) None

6) External CSS (if needed)

Component Map:



Database Organization

USERS			
INTEGER	id	PK	Auto-increment
TEXT	name		Unique
TEXT	password		For authentication
INTEGER	elo		
DATE	created_at		

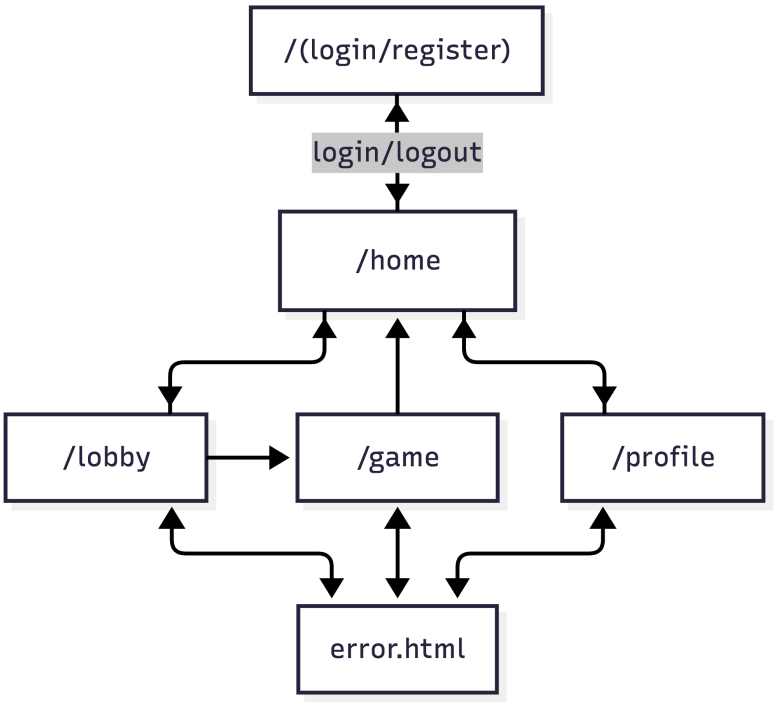
GAMES

INTEGER	id	PK	Auto-increment
INTEGER	winner_id	FK	USERS(id)
DATE	timestamp		

RESULTS			
INTEGER	id	PK	Auto-increment
INTEGER	game_id	FK	GAMES(id)
INTEGER	user_id	FD	USERS(id)
INTEGER	elo_change		

Site Map:

7) Templates (HTML)
    a) **(login.html) /login** (username + password. Error message if login fails).
    b) **(register.html) /register** (username + password. Error message if username is taken).
    c) **(home.html) /home** (List of lobbies + button for users to create their own)
    d) (**lobby.html**) **/lobby/<id>** (Waiting room, displays the users in the lobby and button to leave lobby)
    e) (**game.html**) **/game/<id>** Needs a lot of things:
        - Canvas for drawing
        - Timer for all the phases (drawing, voting)
        - List of players and their current scores
        - Button for voting
    f) (**profile.html**) **/profile** (shows username, elo, etc.).
    g) **error.html** (invalid id’s or permission errors).



Tasks & Assignments

TASK BREAKDOWN

Task	**Devo(s)**
JS & DB stuff	Andrew Tsai
JS & HTML stuff	Owen Zeng
JS & CSS stuff	Zixi Qiao
stuff	Yuhang Pan